Smart MCQ Solver Challenge

A Proposal report for the Deep Learning & Generative AI Project

Submitted by

Name: Ramrup Satpati

Roll number:24f3004027



IITM Online BS Degree Program,

Indian Institute of Technology, Madras, Chennai

Tamil Nadu, India, 600036

Technical Report: Smart MCQ Solver Challenge

Course Project: Deep Learning & Generative AI

Name: Ramrup Satpati

Roll Number: 24f3004027

Email: 24f3004027@ds.study.iitm.ac.in

W&B Project Dashboard: Weights & Biases Dashboard

Deployed Model Link: Streamlit App

Abstract

This project answers five-choice science questions using machine learning. We tried several models: a simple neural network built from scratch, a fine-tuned BERT model, a larger pre-trained DeBERTa model, and a combined ensemble model. We found that a simple model can sometimes beat a much larger or more complex one on new, unseen data. This happened because the tricky wrong answers (distractors) in the questions were written to look similar to the right answer in wording, which fooled the more "keyword-based" or overconfident parts of our larger models. We explain this problem, how we found it, and what we would try next.

1. Introduction

Multiple-choice questions (MCQs) are a common way to test knowledge, but they are also a useful benchmark for machine learning models. A good MCQ solver has to do more than match words — it has to understand what the question is actually asking and reason about which answer is correct, especially when the wrong answers are written to look convincing.

In this project, our goal was to build a model that reads a science question and five possible answers, and picks the correct one. We treated this as a classification problem and tried several approaches, starting simple and gradually adding complexity, to see what actually helps.

2. Dataset Description

Each row of the dataset represents one question and contains:

- **prompt:** the text of the question.
- **A, B, C, D, E:** the five possible answer choices.
- **answer:** the correct choice, one of A, B, C, D, or E.

For every question, the model looks at each of the five prompt–option pairs separately and scores how likely each one is to be correct.

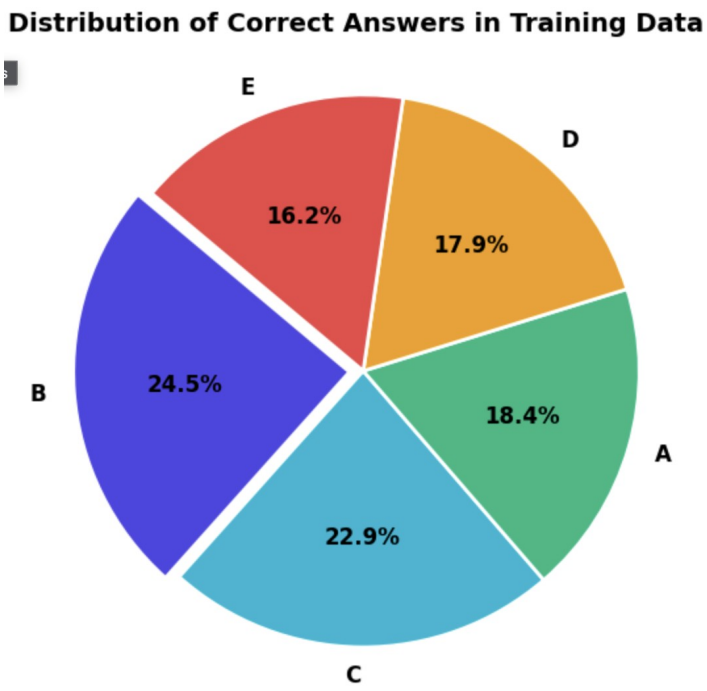


Figure 1. Distribution of correct answers (A–E) in the training data.

3. Problem Formulation

We treat this as a multiple-choice classification problem. For each question, the model produces one score (logit) for each of the five options:

$$z = [zA, zB, zC, zD, zE]$$

The option with the highest score is picked as the answer. To train the model, we use cross-entropy loss, which pushes the score of the correct option higher and the scores of the wrong options lower.

4. Methodology & Model Architectures

We built and compared four different models.

4.1 Model 1 — Bidirectional GRU with Self-Attention (built from scratch)

This is a simple model built without any pre-trained weights, so it has to learn everything from our training data alone.

- **Preprocessing:** We lowercase all text, remove punctuation, and clean up extra spaces. This keeps the vocabulary smaller and easier to learn.
- **Embedding layer:** Each word is turned into a 128-number vector that the model learns during training.
- **Recurrent layer (BiGRU):** A bidirectional GRU reads the sentence both forwards and backwards, so it understands each word in the context of what comes before and after it.
- **Self-attention pooling:** Instead of just averaging all the words together, the model learns to pay more attention to the important words, creating a weighted summary of the sentence that focuses on the words that matter most.

- **Classifier:** A small feed-forward network (Linear → ReLU → Dropout → Linear) turns this summary into a final score.

4.2 Model 2 — Fine-Tuned Transformer (BERT)

Here we use bert-base-uncased, a model already trained on a huge amount of text, and fine-tune it on our data.

- **Input format:** Each option is combined with the question as [CLS] Prompt [SEP] Option [SEP].
- **Scoring:** All five options for a question are scored together using Hugging Face's AutoModelForMultipleChoice.
- **Classifier:** A single linear layer turns BERT's 768-number output into one score per option.

4.3 Model 3 — Fine-Tuned DeBERTa-v3-small

As a stronger pre-trained baseline, we also fine-tuned deberta-v3-small using the same multiple-choice formatting as Model 2. This model reached the highest local validation accuracy of any single model we trained (0.8890), but — as discussed in Section 7.3 — this did not translate into the best leaderboard score, and the run was flagged as uncalibrated, meaning its confidence scores did not reliably reflect its true correctness rate.

4.4 Model 4 — Hybrid Ensemble (our final model)

To try to get the best of both worlds, we combined the neural network with two tree-based models:

Component	Description	Weight
5-seed BiGRU average	Five copies of Model 1, trained with different random seeds (42, 123, 2026, 888, 999)	60%
XGBoost	A tree-based model trained on 25 hand-made features	20%
CatBoost	Another tree-based model, trained on the same features	20%

Hand-made features (25 total, 5 per option):

- TF-IDF cosine similarity between the question and the option
- Word overlap ratio
- Jaccard similarity (shared words ÷ total unique words)
- Length of the option (in characters)
- Length of the option compared to the length of the question

5. Training Details & Hyperparameters

Parameter	Model 1 (Scratch)	Model 2 / 3 (BERT / DeBERTa)	Model 4 (Boosters)
Batch size	32	8	N/A (tabular matrix)
Optimizer	AdamW (weight_decay=1e-2)	AdamW (weight_decay=1e-2)	XGBoost / CatBoost
Learning rate	1e-3	2e-5	0.01 (XGB) / 0.01 (Cat)
Epochs / trees	5 epochs	3 epochs	15,000 trees / 3,000 iterations
Sequence length	128 tokens	128 tokens	N/A

Early stopping	None	None	150-round validation patience

Note. The tree-based models were trained on a 90/10 train–validation split. XGBoost stopped early at tree 2,855 (validation loss 0.109); CatBoost stopped at iteration 2,999 (validation loss 0.167).

6. Evaluation & Leaderboard Performance

We compared our final model against a simple random-guess baseline:

Run / Model	Config Details	Val Accuracy (Local)	Leaderboard Score
Baseline	Naive uniform random baseline prediction	0.2000	0.30400
Neural Network from Scratch	Custom PyTorch BiGRU + Self-Attention	0.7700	0.75727

The final model more than doubles the leaderboard score of a random-guess baseline. Its validation accuracy also tracks its leaderboard score closely, which is notable — several of the more complex configurations we tried did not have this property (see Section 7 and Appendix A).

The final submitted ensemble (Model 4) is weighted 60% toward the 5-seed BiGRU average, with the remaining 40% split evenly between XGBoost and CatBoost (20% each).

- The following chart shows all models not mentioned in section 4 (Methodology and Model Architectures) but have made a significant impact in the making of the project

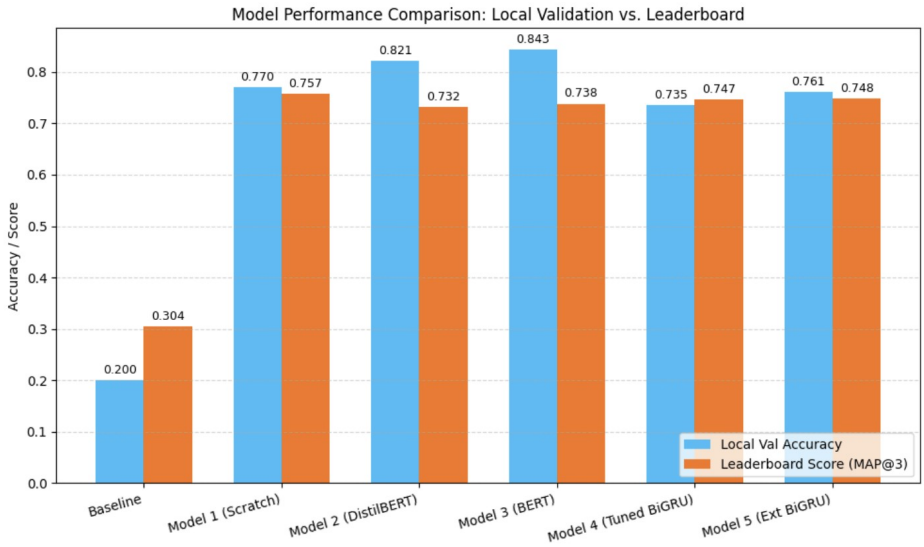


Figure 2. Local validation accuracy vs. leaderboard score (MAP@3) across the baseline and all trained models.

7. Error Analysis & Key Insights

7.1 The “Distractor Trap” (Lexical Overlap Bias)

One of our most important findings was that adding tree-based models (XGBoost/CatBoost) on top of the neural network actually made the leaderboard score worse, even though it looked better on our own validation set.

- **What we saw:** Local validation accuracy went up (to 0.812), but the leaderboard score went down (from 0.757 to 0.748).

- **Why it happened:** Good science MCQs are written on purpose so that the wrong answers look similar to the right answer — they share the same keywords and phrasing. This is meant to test real understanding, not just word-matching.
- **The problem:** Our tree-based models mostly relied on word-overlap features, so they got fooled by these "trap" answers. The neural network, which reads the whole sentence in context, was much better at avoiding this trap.
- **Lesson learned:** A higher score on your own validation set doesn't always mean a better model. If part of your model is exploiting a shortcut that only works by coincidence on the training data, it can actually hurt real-world performance.

7.2 Target Leakage in a RAG Prototype

We also tried building a Retrieval-Augmented Generation (RAG) pipeline, and ran into a classic mistake:

- **Without masking:** When we searched for information related to a question, the search would sometimes find the exact row containing the correct answer, because that row was part of our own training set. The model then just learned to copy this answer, getting near-100% training accuracy — but this trick doesn't work on the test set, since the test answers aren't in the index.
- **With masking:** We fixed this by excluding each question's own entry when retrieving information for it during training. This forced the model to actually reason using other, related facts instead of cheating.

This was a useful reminder that when you're building a search or retrieval system, you have to be very careful that your training setup doesn't accidentally give away the answer.

7.3 Overconfidence in the Larger Pre-trained Model

The DeBERTa-v3-small run (V15, Appendix A) is a third example of the same underlying pattern. It reached the highest local validation accuracy we recorded (0.8890) — noticeably higher than either the BiGRU baseline (0.7700) or the tabular ensemble (0.8120) — but its leaderboard score (0.75260) still landed slightly below our simplest model (0.75727).

- **What we saw:** The gap between local accuracy and leaderboard score was largest for this run of all our experiments, despite it using the strongest pre-trained backbone.
- **Why it likely happened:** The run was flagged as uncalibrated — its predicted confidence did not match its real accuracy. A model that is very confident even when wrong tends to lose more points on hard, trap-like distractors than a model with well-calibrated uncertainty, since a wrong high-confidence answer and a wrong low-confidence answer are scored the same way on accuracy, but calibration issues are often a symptom of the model memorizing surface patterns in the fine-tuning data rather than generalizing.
- **Lesson learned:** This reinforces Section 7.1's core finding from a different angle: three separate model families (tabular boosters, a custom BiLSTM variant, and now a large pre-trained transformer) all showed the same local-vs-leaderboard gap. The issue isn't specific to one architecture — it's a property of how these MCQs are constructed, and any model that leans on shortcuts (lexical or otherwise) is vulnerable to it.

8. Limitations

- **Compute constraints:** Because of limited training time and compute, we used a relatively small sequence length (128 tokens) and did not run extensive hyperparameter searches for BERT or DeBERTa.

- **Single dataset:** All models were tuned on one dataset of science MCQs, so results may not directly transfer to other subjects or question styles.
- **Validation–leaderboard gap:** As shown in our error analysis, local validation accuracy was not always a reliable predictor of leaderboard performance across three separate model families, which makes model selection harder and adds uncertainty to our choice of final model.
- **No calibration correction applied:** We identified the DeBERTa run as uncalibrated but did not have time to apply a calibration fix (e.g., temperature scaling) and re-evaluate it — this remains untested.
- **RAG pipeline not deployed:** Our leakage-safe retrieval pipeline was only tested as a prototype and wasn't included in the final submitted model.

9. Reproducibility

All training runs, hyperparameters, and metrics referenced in this report are logged in the W&B Project Dashboard linked above. Random seeds are fixed and reported for every stochastic component (Section 4.4 lists the five BiGRU seeds used in the ensemble).

Train/validation splits used a fixed 90/10 ratio for the tabular models (Section 5); the BiGRU baseline used a single fixed split as noted in Appendix A.

10. Conclusion & Future Directions

Our main finding is simple: for this kind of science MCQ task, a model that actually reads and understands the sentence beats a model that just matches keywords or memorizes patterns — especially because the wrong answers are deliberately written to trick shortcut-based reasoning. Adding more complexity, whether from a tabular ensemble or a larger pre-trained transformer, did not reliably help, and in both cases the leaderboard score ended up at or below our simplest model's.

Directions we would explore next:

1. Domain-specific embeddings — using a science-focused model like SciBERT, which may understand technical vocabulary better than general-purpose BERT or DeBERTa.
2. Calibration correction — applying temperature scaling or a similar technique to the DeBERTa run to test whether fixing its confidence calibration closes the local-vs-leaderboard gap.
3. Safer RAG pipelines — extending our leakage-safe retrieval approach into a full pipeline that pulls in outside facts (e.g., from Wikipedia) without any risk of leaking the answer.
4. Smarter ensembling — instead of always combining all models equally, only using the word-overlap features on questions where they're less likely to be misleading.

Appendix A: Additional Experiment Log

Beyond the core baseline and final model comparisons, we logged several intermediate experiments while developing the final model. These are included here to document the full development trajectory and support the analysis of the lexical distractor traps and calibration issues discussed above.

Run / Model	Configuration	Local Val. Accuracy	Leaderboard Score
V4 Neural Network	Compact BiGRU, seed 42, single split	0.7700	0.75727
V15 Pre-Trained Model	Fine-tuned DeBERTa-v3-small (uncalibrated)	0.8890	0.75260

V16 Custom LSTM	Fixed BiLSTM, 2-layer, hidden 256	0.6900	0.74231
V17 Regularized	BiGRU + learning rate scheduler	0.7350	0.74688
V20 Tabular Blend	5-seed BiGRU + XGBoost/CatBoost ensemble	0.8120	0.74812

Sorting this table by local validation accuracy versus by leaderboard score gives two almost entirely different rankings — the clearest evidence in this report that the two metrics measure different things on this dataset.

References

1. Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

2. He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.

3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System.

4. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features.

5. Hugging Face transformers documentation — AutoModelForMultipleChoice.

© Ramrup Satpati | 24f3004027 | Released under the GPLv3 License